BENGALI WORD ASSIMILATION INTEGRATING USING DEEP LEARNING METHODS

By

1. Anindya Halder ID: 200101010
2. Mst. Faujia Akter ID: 200101035
3. Most. Joiariya Zannath ID: 200101065

B.Sc. Engineering in Computer Science and Engineering

Department of Computer Science and Engineering

Faculty of Electrical and Computer Engineering

Bangladesh Army University of Science and Technology (BAUST)

DECEMBER 2023

Bengali Word Assimilation Integrating Using Deep Learning Methods

This thesis is submitted in partial fulfillment of the requirement for the degree of B.Sc. Engineering in Computer Science and Engineering

By

1. Anindya Halder ID: 200101010
2. Mst. Faujia Akter ID: 200101035
3. Most. Joiariya Zannath ID: 200101065

Supervised by

Fatema Tuz Zohra

Assistant Professor, Department of CSE

Department of Computer Science and Engineering

Bangladesh Army University of Science and Technology (BAUST)

Saidpur Cantonment, Nilphamari, Bangladesh

The thesis titled **"Bengali Word Assimilation Integrating Using Deep Learning Conciliar"** submitted by ID. 200101010, 200101035 and 200101065 Session 2019-2023 has been presented on 19 / 12 / 2023 and accepted as satisfactory in fulfilment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering (CSE) as B.Sc. Engineering to be awarded by the Bangladesh Army University of Science and Technology (BAUST).

Board of Examiners

1. _____ Supervisor

   Fatema Tuz Zohra

   Assistant Professor

   Department of Computer Science and Engineering (CSE)

   Bangladesh Army University of Science and Technology (BAUST)

2. _____ Member

   Hasan Muhammad Kafi

   Assistant Professor

   Department of Computer Science and Engineering (CSE)

   Bangladesh Army University of Science and Technology (BAUST)

3. _____ Member

Ashrafun Zannat

Lecturer

Department of Computer Science and Engineering (CSE)

Bangladesh Army University of Science and Technology (BAUST)

_____

Dr. S.M. Jahangir Alam

Professor and Head

Department of Computer Science and Engineering (CSE)

Bangladesh Army University of Science and Technology (BAUST)

CANDIDATE'S DECLARATION

It is hereby declared that the contents of this project are original and any part of it has not been submitted elsewhere for the award of any degree or diploma.

1.	Anindya Halder	200101010	
2.	Mst. Faujia Akter	200101035	
3.	Most. Joiariya Zannath	200101065	

ACKNOWLEDGMENTS

First of all, We are grateful to almighty Allah who enabled me to complete this thesis successfully. Thereafter, our sincerest thanks and gratitude to our honorable thesis supervisor Fatema Tuz Zohra who is an Assistant Professor in the Department of Computer Science & Engineering (CSE) at Bangladesh Army University of Science & Technology (BAUST), Saidpur. Her inspiration has been stimulated me to complete our thesis.

We would like to express our special thanks to Professor Dr. S.M. Jahangir Alam Head of the Department of Computer Science & Engineering (CSE) for his valuable suggestions, positive advices, encouragement and sincere guidance throughout our thesis work. We also convey my special thanks and gratitude to all of the respected teachers of the department of CSE including other departments.

We sincerely thank to respected honorable Vice Chancellor sir and respective staffs, Chief of Army and staffs, board of trustees and all members including academic members, and so on,those who led this institution and made me a bright future.

We would like to thank all of our friends and the staffs of the department for their valuable suggestions and assistance. Finally, we would like to thank our parents and our family for their steady love and great support during our study.

ABSTRACT

Natural language processing makes heavy use of topic modelling approaches to derive subjects from unstructured text input. The Bangla language in the 7th most spoken language in the world. English is the predominant language for technical knowledge and online resources, journals and documents. However, Bangla's NLP (natural language processing) technology is not as advanced as English's, and not much research has been done about the Bengali dialect one of the most widely spoken languages in the world in its context. Because of this, it's an opportunity to deal with this problem in order to handle and arrange information effectively. A Bangla phrase from a story might be something like this: an aggressive, inquisitive, essential optative, or exclamatory written document. Using the information in the data set, a number of machine learning (ML) and deep learning (DL) methods are used to classify the sentence in the text document. Our dataset is distinct because it was manually assembled while maintaining consideration for the origins and sentence structure of Bangla. The dataset is totally occupied by the raw linguistic ordinals of 16053 sentences, which was the preprocessed and severely cleaned from various unnecessary inputs. We used a supervised Deep Learning model that includes the following: Bidirectional Encoder Representations from Transformers (BERT) AND Long Short Term Memory (LSTM), Bidirectional Long Short-Term Memory (BiLSTM), Convolutional Neural Network (CNN), Gated Recurrent Unit (GRU). Collectively, these techniques produced the final summary. The accuracy, recall, and F-measures of the suggested model were assessed in comparison to several human made summaries. Furthermore, this study's limitations and potential implications have been explored.

Keywords: Convolutional Neural Network(CNN), Gated Recurrent Unit(GRU), Long Short-Term Memory(LSTM), Bidirectional Long Short-Term Memory(Bi-LSTM), Deep Learning.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

Introduction

1.1 Overview

One of the most extensively spoken languages in the world is Bengali. Over 98% of Bangladeshis plus a sizable percentage of Indians speak this language as their mother's tongue. A lot of individuals prefer digital paper as it can be viewed on laptops, mobile phones, and other gadgets used in the ever expanding realm of social media. Because of this, there are numerous Bangla news websites that offer the most recent information on a variety of subjects. A number of news based websites are also built on various online platforms like Facebook and Instagram. Every online Bangla news provider has a different layout, style, and categorization system to convey the news items. A lot of people who read newspapers enjoy reading and evaluating news from different sources. These varied portals generate content for almost every categorization, thus it might not always be possible to classify these enormous collections of material into the appropriate categories. Online news websites include subject categories and subcategories, which vary significantly across newspapers. These might not be enough to pique readers' curiosity as a result. Because readers like to research news from several sources instead of just one, providing them with relevant news depending on its contents may improve their experience. The article's main objective is to help Bengali internet news readers find relevant material by using multi category classification. Our main objective is to construct a computerized framework for the vital task of classifying Bangla reports based on their substance, which is important to readers. Over the past few decades, many methods from machine learning and statistics have been applied to extract meaningful features and efficiently classify textual data. These supervised learning techniques require a lot of textual data to be trained. There are many large datasets available in many other languages, but only a small number of procured datasets have been developed

specifically for the classification of documents in Bangla. This has led to a paucity of works addressing the problem of classifying Bangla texts. But most of these investigations used very small datasets, few thousand data, which is not enough to train a model that is supervised. Due to a lack of resources, learning the language of Bangla is very difficult. Very few libraries are available for preprocessing data. The Bangla language has a large number of compound words, or deep thongs. Additionally, by making use of some of the distinctive characters found in the Bengali alphabet, a single stem or root phrase in Bangla can be used to create a significant number of meaningful words. Conversely, a single word can have multiple implications depending on the sentence or context in which it's used. These facts significantly increase the difficulty of preprocessing Bengali sentences. Because of this, preprocessing Bangla articles requires a significant amount of computer time. To answer the user's query, we aim to classify the contents in Bangla news articles. Our general method for fine tuning the BERT model that has been pretrained comprises three stages: first, pretrain BERT using within task or in domain training data; second, if numerous related tasks are available, optionally finetune BERT using multitask training data; and third, fine tune BERT according to the intended task. We also investigate layer selection, catastrophic omitting, preprocessing of long texts, low shot learning problems, and layer wise rate of learning fine tuning methods for BERT on target tasks. In this article, we address the previously mentioned problems and define Bangla news contents using a large dataset.

1.2 Background and Present State

There is plenty of data and information about news and events in the modern world. Information is easily accessible to people nowadays thanks to modern communication technology. Data accessibility is not an issue, but it becomes risky when the data contains false information or stories that go viral globally. This fake news has an immoral impact on social and personal relationships and has the potential to cause misunderstandings and even political tension. According to a recent survey, roughly one in three people in the US, Spain, Germany, the UK, Argentina, and the Republic of Korea say they have come across false or misleading information about COVID-19 on social media [1]. The effects of fake news are wreaking havoc on the world. In Bangladesh, sev-

eral pagodas were set on fire in 2012 after a picture appeared that depicted the Quran in an offensive way. The young Buddhist who was tagged in the picture was not in any way associated with the picture. Even though the identity of the image could not be confirmed, many rumours circulated as a result of it [2]. A false rumour about the Padma Bridge's officials giving up people's lives at the building site went viral online in 2019. This rumour then raises the possibility that strangers are abducting children [3]. An example of this can be found in Bangladesh in the Ramu incident of 2012, when nearly 25,000 people joined forces to abolish Buddhist temples based on a post on Facebook from a fictitious account [4]. Thankfully, there are websites dedicated to combating fake news. However, these websites are unable to adequately address the majority of fake news incidents. Certain computational techniques are employed to separate false information, which is detrimental to our day-to-day existence. There is a lot of research being done on English languages these days. Currently, over 341 million people use Bangla to communicate worldwide. However, there is no tool to identify false information written in Bangla. Naturally occurring language processing tasks for regional languages are challenging because of these incredibly low resources. Regional language assets are therefore still needed for a variety of tasks related to natural language processing. However, some of the regional tongues have a history dating back over a millennium. The reach of such regional languages on digital platforms is imperative, as their presence is crucial. There are 780 dialects and languages in India, according to the People's Linguistic Survey of India. 150 of them might disappear over the next fifty years [5] [6]. 191 Indian languages have been classified as vulnerable by UNESCO. The authors of [7] suggested a fuzzy rule deduction structure for the the Bangla language web text classification in order to classify Indian languages, and they were able to achieve a 98.63% classification rate. Since Bangla is another regional language of India with fewer resources, the authors created a collection of 10,300 articles divided into 8 categories on their own. Likewise, Dhar et al. [8] presented a novel feature selection technique for Bangla text categorization called TF IDF-ICF (Term Frequency Inverse Record Frequency Inverse Class Frequency), with a 98.87% accuracy rate. By using a Multi Layer Perceptron (MLP) and dimensionality reduction methods, additional authors in [9] [10] achieved 98.03% and 97% of accuracy on 4000 and 1960 Bangla documents, respectively.

1.3 Problem Statement

The sentence Scoring is an evaluation method that identifies the most pertinent sentence within a passage by using a similarity-based ranking model. Tokenizing every sentence in the training dataset, this model creates sentence vectors from them. Here, we employed a word2vec model that had previously been trained on the Bengali information dataset. We then adjusted the model using our dataset, and the result was a summary. In this case, a vector represents each sentence. Subsequently, every vector is compared with every other vector that is included in the written material, and the cosine equivalent technique is used to determine each vector's similarity score. A method for identifying the entities with names from a string of phrases or a sentence is called Named Entity Recognition (NER). Finding the person, place, organisation, and challenge in the passage sentence is the main goal here. There are multiple methods available for NER tagging. In this case, the named entities have been tagged using the BIOES technique. The relationship between the words in a sentence is indicated by their parts of speech. We have employed a pre-trained CRF based model for Parts of Speech (POS) tagging, created by Sagor Sarker, to identify POSes like nouns, pronouns, adjectives, as well as verbs in a sentence.

1.4 Objectives

Our work focuses on categorizing a dataset using a variety of deep learning and machine learning algorithms, such as CNN, GRU, BiLSTM, LSTM, and BERT. The primary aim is to assess their performance using metrics like precision, recall, F1 score, as well as through visual aids like paradigms and diagrams. This evaluation will help determine which algorithm yields the most proficient results. By comparing the outputs of these algorithms, you'll be able to discern which one performs best. This assessment isn't solely about the accuracy of the categorization but also involves evaluating how well these algorithms handle the training and testing data, shedding light on their adaptability and generalization capabilities. Our study design is comprehensive, encompassing a range of contemporary algorithms used in natural language processing (NLP). The utilization of various evaluation metrics will provide a nuanced understanding of their

strengths and weaknesses in document categorization. This holistic approach allows for a more informed conclusion about which algorithm might be most suitable for this specific task.

1.5 Issues Encountered

We face some problem when we working on a project involving Bengali word assimilation integrating deep learning methods. Our main challenge is to collect dataset. We collect 16053 data for our dataset and we collect it manually it was a big challenge for us. We collect our data from social media platform Facebook. But nowadays Bengali text is rarely used in the online world. English typing is used in most of the vagaries, so it is very challenging for us to collect 16053 data.

1.6 Social Impact

Our topic is integrating Bengali word assimilation deep learning methods. Its main purpose is to classify Bengali sentences. Our work will not have any negative impact on the society. We can categorize through our work what type any comment is. We divided our dataset into 7 class levels threat, religious, insult, troll, normal, heat, vulgar. Nowadays, people make many good and bad comments on social media. By our work er can classify that positive and negative comment and be aware about it. Our work will not have any negative impact on the society but will bring benefits to the society.

1.7 Scopes and limitations

As mentioned previously the limitation in the work of NLP regarding the Bengali or any relative language is that the lack of data available for the research enthusiasts. As we venture forth to unravel or try to get toe to toe with the evolution of recent decades of upbringing of technology, we come face to face with the challenge of the missing jigsaw puzzle pieces. As we can see the wholesale of English turbulent data informative, all we can do is start from the scratch again. But it also opens a certain scope to leave a mark for the generation of Artificial intelligence and data science. This field is full of

scopes to open potential assurances towards acquiring matrix in the computer science retrospect.

1.8 Organization of the Thesis

The thesis is meticulously organized into distinct chapters, systematically addressing various critical aspects of the research. Beginning with the Literature Review, Chapter 2 provides an extensive overview of existing knowledge, defining the problem, proposing solutions, and summarizing related applications while outlining open issues. Chapter 3, Methodology, details the chosen approach, algorithms used, data preprocessing, and variations in model implementation. Following this, Chapter 4, Implementation, focuses on the practical application, outlining the strategy, detailing the process, and specifying instrumental requirements. Result and Analysis, Chapter 5, delivers outcomes, quantitative analysis, individual algorithm results, experimental findings, and a comparative performance analysis. Finally, Chapter 6, Conclusion and Future Work, synthesizes conclusions, recommends future research, notes limitations, and suggests potential research directions. This sequential arrangement offers a comprehensive progression from literature review to implementation, analysis, and implications for future research.

1.9 Summary

As to come to conclusive explaining we are trying to contribute to the vast ocean of the natural language processing research world, with a exemplary output of counter procedure of variation finding in the dataset made by us. There were previously such work done to a extent, but what we are trying to convey is to put another small part in the criteria. As students of a such intrusive field our main goal is to beyond the vails of our limitations and accept the reward that comes to be an achiever to contribute in our glory field.

CHAPTER 2

Literature Review

2.1 Overview

This chapter describe the basic concept of the Bengali word assimilation integrating deep learning method. The majority of research on the English language has been conducted because it is an international language. However, study on the language of Bangla language also exists. A proposal for an agrarian a brief from a passage was put forth by Kamal et. al. [11]. After building the model in three main stages, he was able to use the model to obtain an agrarian summary in an article that was generated by a machine. This paper selects sentences for summary based on the distances and parallels in each sentence's neighbourhood, which are computed automatically from an automatically generated paragraph chart from every record. Regional Indian languages are aesthetically rich, less resource-rich, and agglutinative by nature [12]. Therefore, in comparison to English natural language, developing intelligent systems for real-time application is quite challenging. Research on automatic news classification is very popular.

2.2 Problem Statement

Develop a compact, deep learning-based solution for effortlessly integrating foreign words into Bengali, guaranteeing linguistic coherence and cultural relevance. Create a user-friendly application that accurately assimilates words from multiple languages into Bengali, using a lightweight deep learning model.

Cultural Coherence: Make sure assimilated terms follow linguistic and cultural conventions in Bengali.

Small Model Design: Construct a tiny CNN that works well in low resource settings.

Instantaneous Adaptability: Give the algorithm the ability to instantly adjust to newly learned terms in real time.

Anticipated Result: A simplified, user-friendly application that offers organic and culturally suitable Bengali translation for foreign terms.

Success Standards: High precision in word assimilation. Compact design with effective functionality. Positive comments from users on cultural significance and language coherence.

2.3 Solution of the Problem

We face some problem when we working on a project involving Bengali word assimilation integrating deep learning methods. Our main challenge is to collect dataset. We collect 16053 data for our dataset and we collect it manually. It was a big challenge for us. It appears that there are some ambiguities or gaps in our statement. Regarding ”Bengali Word Assimilation Integrating Deep Learning Methods,” if you could give us further background information or specifics about the issue we are referring to or what we are looking for help with, we would be pleased to try my best to assist.

2.4 About This Thesis

The ”Bengali Word Assimilation Integrating Deep Learning Methods” thesis stands as a pioneering endeavor seeking to elevate the assimilation of foreign vocabulary into Bengali through the application of sophisticated deep learning techniques. What sets this research apart is its reliance on an expansive and diverse dataset an expansive reservoir encompassing an extensive array of terms from multiple languages in conjunction with Bengali literary works. This vast and comprehensive dataset serves as the backbone for the computer’s training, enabling it to decipher intricate linguistic patterns, grasp the subtleties inherent in Bengali, and adeptly incorporate terms from diverse languages. Its sheer scale and breadth are paramount, providing the computer with access to an immense virtual repository, essentially functioning as an all encompassing resource for fostering profound language understanding.The huge dataset is important since it serves as a comprehensive resource that gives the computer access to a large library.

2.5 Related Available Application

Using a large dataset, the ”Bengali Word Assimilation Integrating Deep Learning Methods” project can be helpful in the following some ways. It can aid in more accurate language translation by computers. Translations may therefore seem more natural if they are made from Bengali to another language or the other way around.[3] Enhancing Virtual Assistants or Chatbots: The project's outputs may improve the intelligence and ability of Bengali chatbots, or virtual assistants, to comprehend and react in a more human-like manner. Apps for Learning Languages: This initiative could improve the efficacy of language learning applications if you're attempting to learn Bengali. It might demonstrate how well-fitting foreign words are into Bengali sentences.

2.6 Open Issues

Using online news articles from Thailand, researchers classified violent and criminal activities into five categories: murder, accidents, burglaries, drug use, and corruption. They also finished their research and selected the best machine learning classifiers, using TF-IDF for feature extraction and Gradient Boosting Machine, Multinomial NB, Random Forest, Multinomial Logistic Regression, KNN, and SVM for classification. The Support vector classifier finished the categorization process with the highest accuracy (79.41 percent). If the data in each class were in harmony, their results could be better [13]. The aim of this study [14] is to determine the underlying problems in the corpus of Bengali news along with categorise the news using similarity metrics.

2.7 Summary

We plan to develop a model in the future that can perform sentiment analysis in Bengali for news-related information and integrate it into the system for scoring. Additionally, we plan to utilise the Dynamic of TF-IDF model that Oleg Barabash et al. proposed [15]. In order to determine whether the summarization performance may be improved further, efforts are also being made to expand the dataset with textbook summaries and apply deep learning techniques.

CHAPTER 3

Methodology

3.1 Overview

In this part of the article, we are going to elaborate the methodology pathway that we followed in order to get the prominent achievable result outcome. The main methodology part consists of three section, data preprocessing, model training and finally output accuracy. As all of our models are going to integrated with the Bengali fonts, we needed to take several preprocessing steps in order to have a clean sophisticated dataset, In which no unorthodox syllable or mark may regard. So that the performance of our models may not be hinder. We Proposed a methodology diagram to be more discrete about our procedure conscience. The algorithms as mentioned earlier are consists of CNN, GRU, BiLSTM, LSTM and BERT.

3.2 Proposed Methodology

The work methodology involves several crucial steps. It commences with manual collection of the dataset from social media platforms like Facebook. Subsequently, data preprocessing takes precedence, recognizing its pivotal role in preparing data for effective deep learning models. Tokenization follows, splitting the dataset into training and testing subsets. The subsequent phase involves model creation, drawing upon: CNN, GRU, BiLSTM, LSTM, and BERT [16]. Each model undergoes training, a fundamental phase to optimize its performance. The work culminates in performance analysis, evaluating the models based on various metrics. This structured methodology ensures a comprehensive approach from data collection to model training and performance assessment essential for deriving meaningful insights into the efficiency of each model in document categorization tasks. Methodology show in Figure 3.1.

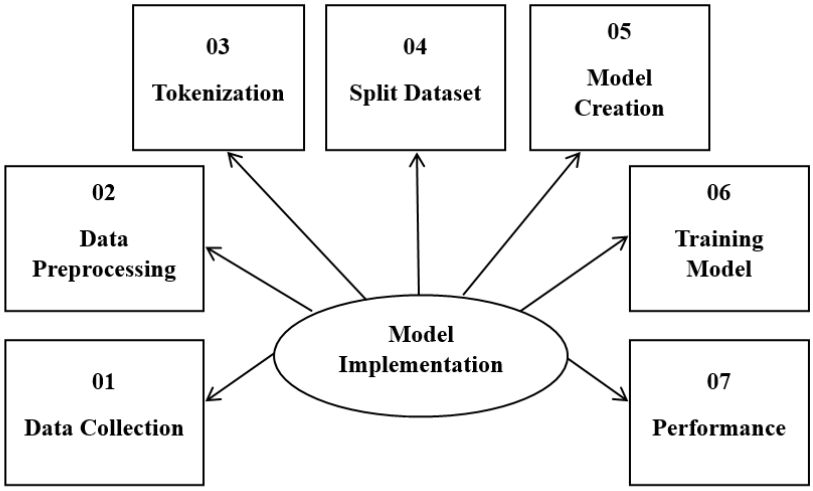

Figure 3.1: The Methodology Diagram.

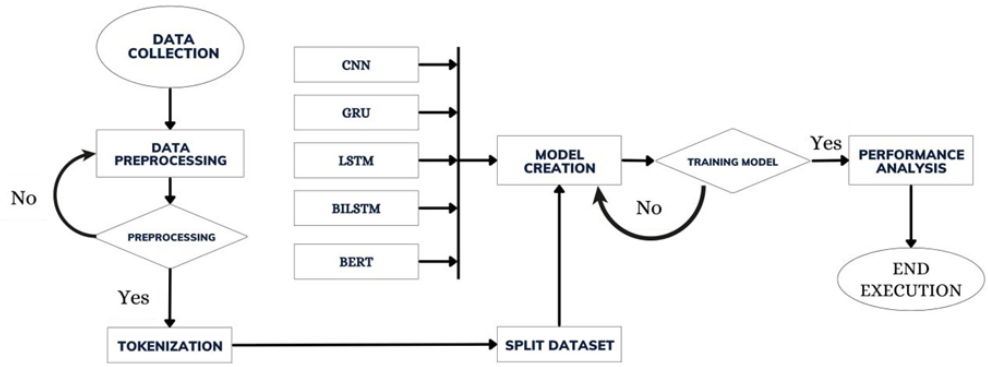

Figure 3.2: Flow Chart Of Our Work.

The schematic delineated in Figure 3.2 outlines a methodical progression adopted within this thesis. Its inception lies in the meticulous collection of data, succeeded by pivotal preprocessing stages designed to refine and enhance the dataset. Following this refinement, the data is segmented into distinct subsets via tokenization, partitioning it into dedicated sets for training, testing, and validation purposes. This partitioned data then becomes the focal point for model development and training. Post-training, an exhaustive assessment of the models' performance ensues, analyzing their efficacy and accuracy. This systematic and stepwise approach ensures a coherent evolution, commencing with the initial data gathering, traversing through model construction via training, and

culminating in a comprehensive evaluation specifically tailored for the efficient categorization of documents

3.3 Summary of The Algorithm

Table 3.1: Comparison of Machine Learning Algorithms

Algorithm	**Type**	**Main Uses**	**Advantages**	**Disadvantages**
CNN	Feed forward	Image and text analysis	Captures spatial hierarchies in data	Limited sequential data processing
GRU	Recurrent	Natural Language Processing	Fewer parameters than LSTM	May suffer from vanishing gradients
LSTM	Recurrent	Text generation, Speech recognition	Handles long-term dependencies	More complex than GRU
BiLSTM	Recurrent	Sentiment analysis, named entity recognition	Captures contextual information bidirectionally	Higher computational requirements
BERT	Transformer	Natural Language Processing, text classification	Captures bidirectional context comprehensively	Computationally intensive during training

3.4 Data Preprocessing

First, we performed a statistical examine of the Bangla text in order to prepare our unbalanced dataset of Bangla News articles over grouping in this section (data acquiring it, data cleaning, and data analysis). Next, we use TF-IDF vectorizer in conjunction

with new feature text data to convert the information to sequence in order to extract features. Dividing the dataset into half, and then using a sampling technique on the training dataset to balance. In the end, we used every classifier we had and determined which performed the best when it came to performance analysis. Finally, we use LIME and SHAP, two explainable AI techniques, to interpret the top model. The process of converting unclean information into a dataset with no contaminants is referred to as cleaning data. Multiple document filters were used in this experiment, along with tokenization, punctuation removal, stop words, and redundant special characters, all of which assisted in number sorting of the categories [17]. The process of breaking up a text document into smaller pieces known as tokens is known as the process of tokenization Furthermore, tokens may be sub-words, individuals, or phrases. Space was used in the present study to keep all worlds apart from the content. Unwarranted particular characters, symbols, numbers, etc. are present in raw text data[14]. Here, the superfluous items have been eliminated using regular expressions. In order to make the data more readable, extraneous symbols and punctuation, such as [., "'—'()!], have been eliminated from the dataset. Every symbol was replaced with a space in order to break down the claim into its component words. In any written data, words with a high frequency are prevalent. They are present in every content category, but they have no such impact on the content itself. These words are referred to as "stop words." These are a few instances of stop words: " ", " ", " ", etc. To obtain more precise models, all those words that stop have been removed from the text. Then the data was differentiated in the labels as religious, threat, troll, hate, vulgar, insult. As shown in the label columns shown in Figure 3.4,

As we also measured the sentence Length in order to understand the datasets caliber better, a diagram of the sentence length distribution count diagram is shown in 3.3.

3.5 Model variation Algorithm wise

In this part we are going to give a brief explaining of the algorithms we used for our categorization procedure part by part.

- **Convolutional Neural Network (CNN):** The workings of a live organism's visual cortex are analogous to those of a neural network made up of convolutions.

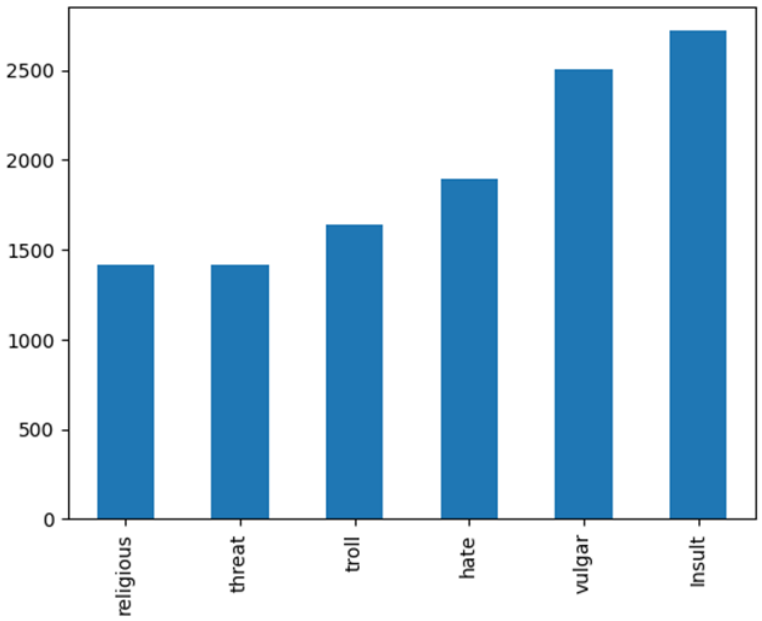

Figure 3.3: Data Labeling Diagram.

Convolution neural networks are an efficient tool for text classification tasks. Pixel values are replaced with word vectors as a matrix in the image classification criteria, which is the only distinction between them. Convolutional neural networks, or CNNs, are some of the most popular algorithms for machine learning. CNN offers an illustration of how to classify texts, both lengthy and short, in an efficient manner. Convolutional Neural Networks (CNNs) are among the most widely used deep learning models for extracting the best characteristics from textual data. This Convo1D layer, MaxPooling1D layers Flatten layer, and The Dense layer are some of the various layers that make up the CNN model's architecture. The embedding layer is the name of the input layer by layer in CNN. Then, the Convo1D layer and Maxpooling1D layer by layer are used to extract local characteristics from the data and reduce the complexity. The Flatten layer converts the two-dimensional features into vector form for the dense layer that comes after. The fully connected layer with activation function is the main component of the dense layer. ReLU and Sigmoid are the two activation functions used in the CNN algorithm's dense layers. So, with CNN's help, we looked at organising the Bangla language. CNN's hidden layers usually contain convolutional, collecting, completely interconnected, and applying layers. We can show a brief view of how the CNN actually works shown in Figure 3.5.

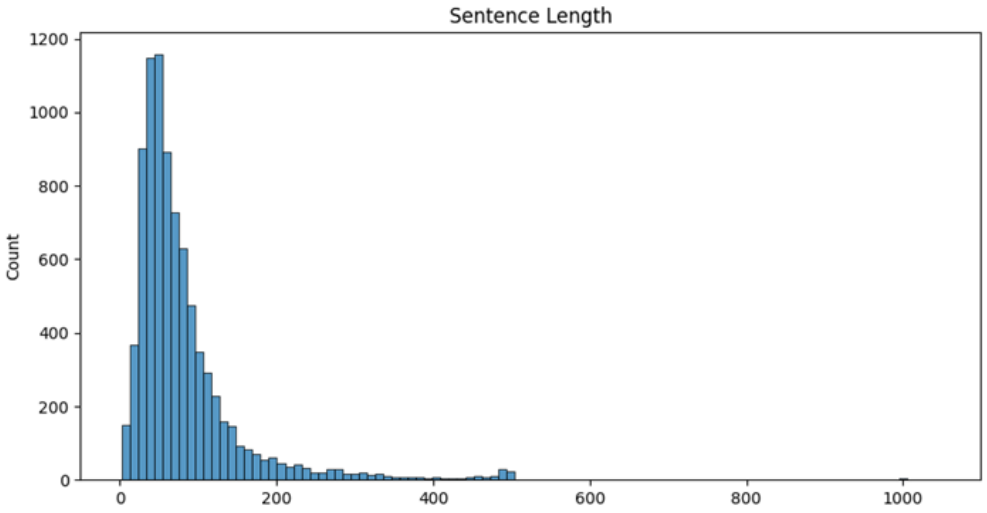

Figure 3.4: Sentence Length Count Diagram.

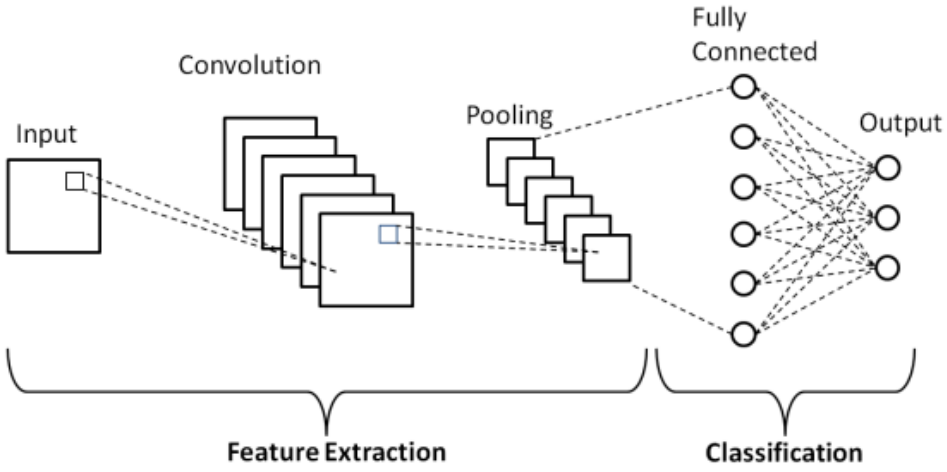

Figure 3.5: Intro To Convolutional Neural Network.

- **Gated Recurrent Unit (GRU):** The RNN was first developed by Goller. Analysing the time series input, the RNN a neural network with memory feedback produces the proper result. The outcome of the RNN cycle process is influenced by the neural network's output at time t, which is fed into the whole thing at time t+1. However, in practical applications, it is found that RNN will show gradient disappearance when evaluating long sequence data, which makes the network, ignore longer term memory and focus only on the last stage of memory education's content. To resolve this issue, many orthodox models have been proposed, including GRU neural networks as well as both short and long recall neural networks. In the below diagram we can see the equations and the GRU. The Gated Recurrent Unit (GRU) is a type of recurrent neural network (RNN) variant that uses gating

mechanisms to control the flow of information within the network. It's designed to address some of the limitations of the traditional RNN, such as the vanishing gradient problem Shown in Figure 3.6.

The update gate z_t and the reset gate r_t in GRU are computed as follows:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

where

- x_t is the input at time step t,
- h_{t-1} is the hidden state at time step $t-1$,
- W_z and W_r are weight matrices for the update and reset gates respectively,
- σ is the sigmoid activation function,
- $[h_{t-1}, x_t]$ denotes the concatenation of h_{t-1} and x_t.

The candidate hidden state $\tilde{h}_t$ is computed as:

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$$

where

- W is the weight matrix,
- $\odot$ denotes element-wise multiplication.

Finally, the new hidden state h_t is computed as a linear interpolation between the previous hidden state h_{t-1} and the candidate hidden state $\tilde{h}_t$, controlled by the update gate:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

This equation describes the flow of information in a GRU unit, allowing it to selectively retain or forget information from previous time steps, making it effective for capturing long-term dependencies in sequential data.

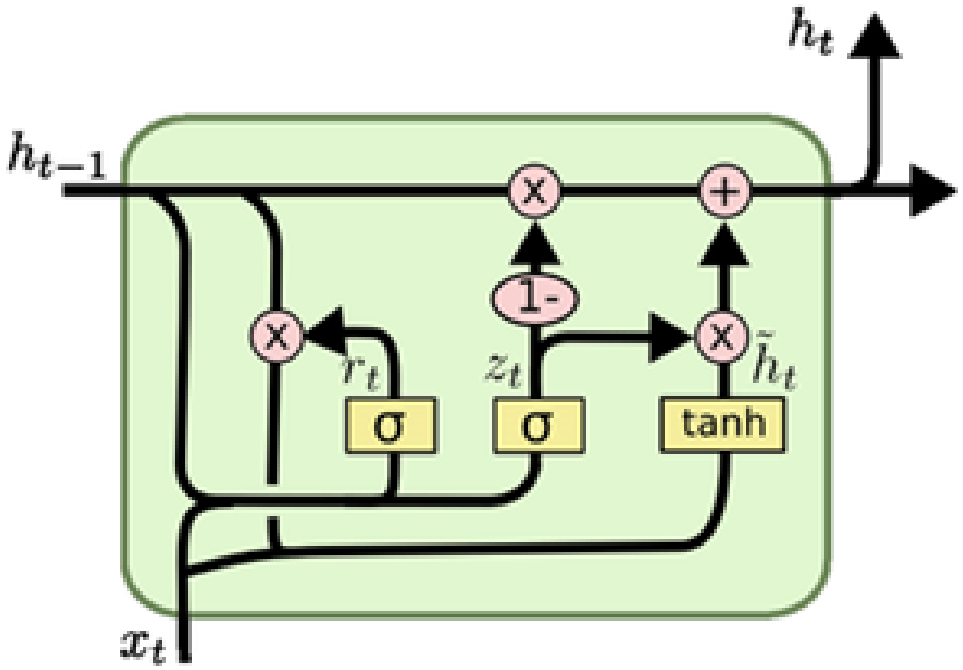

Figure 3.6: Intro To GRU.

- **LSTM (Long Short-Term Memory) and BERT (Bidirectional Encoder Representations from Transformers):** Long short-term memory blocks are used by the recurrent neural network to model how the program receives inputs and generates outputs. The LSTM block is composed of input components such as final outputs, activation functions, inputs from previous blocks, and weighted inputs. The dataset was fed into the following models: a spatial dropout 1D layer (0.4 units), an LSTM model with a 0.2-unit recur rent dropout, a 64-unit dense layer with a RELU activation function, as well as an LSTM model that had an embedding dimensions of 64 and a maximum feature set of 2501. Finally, a final dense layer with six units (based on the number of classes) and the Soft max function. An encoder with 12 Transformer blocks, 12 self-attention heads that are arranged and a hidden size of 768 is part of the BERT-base model. A sequence of a maximum of than 512 tokens is fed into BERT, which outputs a representation of that sequence. The sequence consists of one or two segments. The special classification integrating is contained in the first token of the sequence, [CLS], and the segments are separated by another special token, [SEP]. BERT uses the last hidden state (h) of the first token (CLS) to represent the entire sequence for text classification tasks. For predicting the possibility of label c, a basic softmax

classifier has been included to the highest point of BERT, as shown in Figure 3.7: p(c—h) = softmax(Wh), where W is the specific to the task parameter matrix.

Figure 3.7: Tuning Of The LSTM & BERT.

- **BiLSTM (Bidirectional LSTM):** It is a recurrent neural network, mainly used on Natural Language Processing. It can improve model performance on sequence classification problem and modelling the sequential dependencies between words and phrases in both directions of the sequence. This can provide auxiliary context to the network and result in faster .It integrates the power of LSTM with bidirectional processing, permitting the model to capture both past and future context of the input sequence. The BiLSTM model is more advantageous in some NLP tasks, for example sentence classification, entity recognition and translation. The unrolled BiLSTM is shown in Figure 3.8:

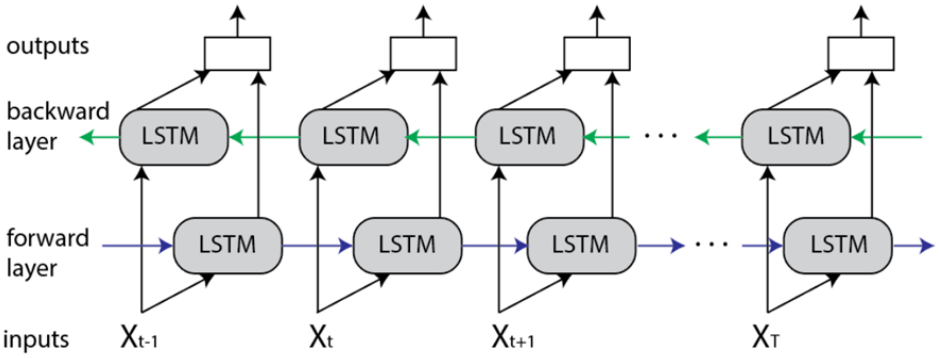

Figure 3.8: Bi-LSTM Model Architecture.

3.6 Summary

Our goal in this work was to develop a system that could categorize in a top level prospect in a manner similar to that of a human. We created an enriched dataset with comments and uniqueness because there weren't enough datasets available in the Bangla language. Sentences were scored by shrewdly combining several methods. We compared the summaries produced by our model with those created by humans in order to assess its performance.

CHAPTER 4

Implementation

4.1 Overview

The stage of the implementation of our thesis, including augmentation, must be accurately reported. This chapter presents the purpose of project our approach to the research the environment setup, the data collection methods, the material used, as well as the manner in which it was used. It defines the target group or subjects of the research and assesses the reliability of the finding. We implement our research by following steps:

- Environment setup
- Data collection
- Fit the model for train

4.2 Implementation

Environment Setup: To finish the project, we used Python 3.8 as the programming language and the Anaconda distribution. The Python library contains a number of machine learning libraries. Python libraries such as NumPy, Seaborn, Matplotlib, NumPy, and Scikit-learn have been used. Pandas was utilised for data manipulation and analysis. Based on Python, Pandas is incredibly quick, adaptable, and easy to use. On the other hand, a wide range of mathematical functions with high-level syntax for working with massive, multifaceted arrays and matrices are supported by the NumPy Python library. A machine learning library for the programming language Python is called Scikit-learn. We made use of Seaborn and Matplotlib for data visualisation.

Data Collection: Data collection is more important for any deep learning project. We collect 16053 data for our dataset. The most important matter is that we collect all data manually. We convert the dataset into seven class level such as religious, threat, vulgar, troll, insult, hate and normal shown in Table 4.1.

Table 4.1: Class Distribution

Class Name	Number of Data
Vulgar	2505
Hate	1898
Religious	1418
Threat	1419
Troll	1643
Insult	2719

Loading Dataset: Load the csv format of our dataset from our Google drive.

Data Preprocessing: Data preprocessing part already discuss in the previous chapter.

Train/Test/Validation Set: This dataset contains two csv files. One is Train set and another is Test set. We have seen that train and test set is equal sized and distribution of class is identical to both of the test and train set. I have used full train set for training and used 20% of the data from test set for validation set and 80% for testing The validation set and the Test set is partitioned into stratified fashion to keep the distribution of class intact.

Tokenization: This dataset contains two csv files. One is Train set and another is Test set. We have seen that train and test set is equal sized and distribution of class is identical to both of the test and train set. I have used full train set for training and used 20% of the data from test set for validation set and 80% for testing The validation set and the Test set is partitioned into stratified fashion to keep the distribution of class intact.

After Tokenization Data visualization: The token can be utilized for approved purposes, such as data analysis, storage, or dissemination, and the original sensitive material is kept private. When a token is used in a transaction or activity, it may, in a secure environment, establish a temporary connection with the original data.

Firstly, a 'Tokenizer' object is initialized with a maximum vocabulary size of 10,000 words and an out-of-vocabulary token to handle words not present in the tokenizer's

learned vocabulary. Next, the ‘ function is applied to the training text data enabling the tokenizer to learn the vocabulary from this data. Following vocabulary creation, the function converts the text data into sequences of integers based on the learned tokenizer vocabulary. This is executed both for the training and test datasets. Subsequently, sequences are padded or truncated to ensure a consistent sequence length. The maximum sequence length is set to 100 (. Sequences longer than this length are truncated, while shorter sequences are padded, both using ’post’ strategy. Finally, the labels for both training and test datasets are transformed into a categorical format suitable for model training using . This converts the labels into one-hot encoded vectors based on the number of classes , which is vital for training classification models. These preprocessing steps, including tokenization, sequence conversion, padding, and label encoding, are essential for transforming raw text data into a format suitable for training machine learning models, particularly those working with text or sequence inputs.

4.2.1 Train model

Convolutional Neural Network (CNN): The code sequence outlines a comprehensive text classification process using a Convolutional Neural Network (CNN) within the Keras framework in Python. It encompasses several crucial stages, commencing with data preprocessing. A ‘Tokenizer‘ object initializes the tokenization process, specifying a maximum vocabulary of 10,000 words (‘max_words‘) and an out of vocabulary token for unseen terms. This tokenizer then learns the vocabulary from the training text data (‘texts_train‘) and transforms the text sequences into integer sequences for both the training and test sets (‘sequences_train‘ and ‘sequences_test‘). These sequences are subsequently standardized to a maximum length of 100 (‘max_len‘) using padding or truncation through the ‘pad_sequences‘ function, ensuring uniform input size for the neural network.

Moving to model construction, the architecture involves layers tailored for text analysis. An embedding layer maps tokenized integers to dense vectors of 50 dimensions (‘embedding_dim = 50‘). A 1D convolutional layer (‘Conv1D‘) with 64 filters and a size of 3, employing Rectified Linear Unit (ReLU) activation, captures local patterns within the sequences. The subsequent global max pooling layer (‘GlobalMaxPooling1D‘) condenses the information for effective feature extraction. The model concludes with a

dense layer (‘Dense‘) utilizing a softmax activation function, facilitating multi-class classification by predicting class probabilities.

The model is compiled using the Adam optimizer with a learning rate of 0.001, employing categorical cross entropy as the loss function and accuracy as the metric for evaluation. Training occurs over 10 epochs with a batch size of 32, utilizing 20% of the training data for validation to monitor the model’s performance.

Post-training, the model is evaluated on the test data, computing loss and accuracy. Predictions are generated for the test set, and comprehensive classification metrics (precision, recall, F1-score) are computed along with a confusion matrix to showcase the model’s performance across various classes. Finally, a heatmap visualization of the confusion matrix is plotted using ‘matplotlib‘ and ‘seaborn‘, illustrating the predicted versus actual classes to provide an intuitive understanding of the model’s predictive capabilities across different categories. Overall, this code encapsulates a robust text classification workflow, encompassing preprocessing, model development, training, evaluation, and insightful result visualization.

Gated Recurrent Unit (GRU): This script encompasses a comprehensive text classification workflow using a GRU-based neural network within Python’s Keras and TensorFlow frameworks, accompanied by essential data preprocessing and result analysis steps. The initial phase involves importing necessary libraries such as NumPy, Pandas, scikit learn, TensorFlow, and Matplotlib. The dataset is assumed to be structured with text data under the ’cleaned’ column and corresponding categorical labels in the ’tag’ column. The data preprocessing pipeline begins by tokenizing the text using a ‘Tokenizer‘ object, considering a vocabulary size of 10,000 words and an out of vocabulary token. Texts are converted into sequences of integers and padded/truncated to a fixed length of 50 tokens, preparing them for model ingestion. Label encoding is employed to transform categorical labels into numerical format for model training. The dataset is split into training and testing subsets using an 80-20 split ratio. A learning rate scheduler function (‘lr_scheduler‘) is defined to adjust the learning rate dynamically during model training, reducing it by half every three epochs to potentially improve convergence. The neural network architecture consists of an embedding layer mapping the tokenized integers into dense vectors of dimension 64. Two stacked GRU layers with varying units and dropout rates are employed for sequential data processing, followed

by fully connected layers with ReLU and softmax activations, respectively.

The model is compiled using the Adam optimizer with a learning rate of 0.001 and categorical cross-entropy as the loss function. It's then trained on the prepared training dataset for 20 epochs with a batch size of 32, utilizing a 20% validation split and employing the defined learning rate scheduler as a callback. Following model training, evaluation on the test set is conducted, computing the loss and accuracy. Predictions are generated on the test set, and a confusion matrix is constructed to visualize the model's performance across different classes. Additionally, a classification report is generated, presenting comprehensive metrics like precision, recall, and F1-score for each class.

A heatmap visualization of the confusion matrix is plotted using 'seaborn' and 'matplotlib', aiding in the intuitive interpretation of the model's predictive performance across various class predictions and actual labels. This script encapsulates a complete workflow from data preprocessing, model construction, training, evaluation, and result visualization for text classification tasks.

Bidirectional Long Short-Term Memory (Bi-LSTM): Building a text classification model using a Bidirectional LSTM neural network within the context of Bengali language text data. It starts by loading necessary libraries, importing datasets, and conducting initial data preprocessing steps. The code meticulously prepares the dataset, ensuring it contains sufficient text length and doesn't have missing values, creating a clean foundation for model building.

The heart of the code lies in constructing the neural network architecture. It employs the Keras Sequential API to assemble the model, beginning with an Embedding layer for converting words into dense vectors. Following this, a Bidirectional LSTM layer is employed, known for its ability to capture context from both past and future tokens. Notably, an additional Dense layer is included, incorporating L2 regularization (with a specified regularization rate) to prevent overfitting by penalizing large weights. This Dense layer is activated using ReLU, aiding in introducing non-linearity to the model. The model is compiled using the Adam optimizer with a specific learning rate and categorical cross-entropy as the loss function, which is well-suited for multi-class classification tasks. The architecture is trained for 25 epochs using the training dataset, with parallel validation on a separate subset. During training, the code visualizes the model's learning progress by plotting the loss and accuracy curves across epochs, facilitating a

deeper understanding of how the model's performance evolves over training iterations. This visual insight helps gauge the model's convergence and performance stability.

The model's performance is thoroughly evaluated. The code calculates and reports accuracy metrics for both the training and testing datasets. Furthermore, it generates a confusion matrix, providing a visual representation of the model's predictions against actual labels across different classes. Moreover, a comprehensive classification report is generated, offering detailed precision, recall, and F1-score metrics for each class, aiding in understanding the model's strengths and weaknesses in classifying Bengali text data.

LSTM & BERT: Beginning with meticulous data preprocessing, including text cleaning and tokenization using the Bangla BERT tokenizer, it meticulously readies the dataset for training, validation, and testing. The model architecture intricately weaves together BERT's contextual understanding with Conv1D and MaxPooling1D layers, culminating in a Bidirectional LSTM layer fortified with attention mechanisms to refine sequence comprehension. The output flows through dense layers with sigmoid activation for multi label classification. Post compilation with binary cross entropy loss and Adam optimizer, the model undergoes rigorous training and validation, meticulously tracking its performance across epochs. Subsequently, it undergoes evaluation on a separate test dataset, gauging metrics like accuracy and loss. Additionally, the code extends its analysis to plotting ROC-AUC curves, assessing the model's discrimination across various classes, and employs LIME explanations to offer insights into the model's decision making process, elucidating its predictive behavior on specific text instances. This comprehensive approach amalgamates cutting edge techniques, presenting a robust solution for complex multilabel text classification tasks in Bengali, backed by thorough evaluation and interpretability measures for deeper insights into the model's functionality.

4.3 Technologies Used in the Thesis

1. **Natural Language Processing (NLP) Libraries:** Frameworks like NLTK, spaCy, or Hugging Face Transformers were employed for text preprocessing, tokenization, and various NLP-related tasks.

2. **Deep Learning Libraries:** TensorFlow or PyTorch were used for implementing neural networks and complex models like LSTMs, BERT, or other architectures.

3. **Data Analysis and Visualization Tools:** Libraries such as Pandas, NumPy, Matplotlib, and Seaborn were utilized for data analysis, manipulation, and visualization.

4. **Machine Learning Models:** Besides deep learning, traditional machine learning models like SVMs, Random Forests, etc. were used for comparative analysis or ensemble methods.

5. **Cloud Computing Platforms:** Google Cloud Platform, AWS, or Microsoft Azure might have been utilized for high-computational tasks or to access specific resources.

6. **Version Control Systems:** Platforms like Git or SVN were used for managing the codebase and versions.

7. **Statistical Software:** Depending on the statistical analysis needed, software like R, SAS, or SPSS could have been employed.

4.4 Summary

The implementation stage which includes setting up the environment, gathering data, and training the model is described in the thesis. NumPy and Scikit learn packages were utilized, together with Python 3.8 running on the Anaconda distribution. The 16,053 hand collected elements in the dataset were converted into seven class levels. Tokenization, visualization, and data preprocessing were done. The CNN, GRU, BiLSTM, and LSTM+BERT models used for model training are all depicted in the paper. Software (Google Colaboratory, VS Studio) are included in the instrumental specs.

CHAPTER 5

Result and Analysis

5.1 Overview

Using a variety of machine algorithms and deep learning classifiers, we classified Bengali comments. The results showed a range of predictions, not all of which were accurate. To assess how well these classifiers are working, we have now employed the Confusion Matrix, Precision, Recall, F1 scores, Accuracy and ROC curve as evaluation matrices. Accuracy, which is simply the ratio of correctly predicted observations to all observations, is the most easily understood performance metric. For the model to be successful, the forecast must be precise.

5.2 Quantitative Analysis and Performance Metrics

In the context of machine learning and deep learning models, several key performance metrics are used to evaluate the effectiveness of a model's training and its performance on unseen data. These metrics include accuracy, validation accuracy, loss, validation loss, and test accuracy [18]. Below, we provide explanations and equations for each of these metrics and emphasize the importance of using them for model evaluation.

Accuracy (ACC) is a fundamental performance metric used to measure the proportion of correctly predicted instances over the total number of instances in a dataset. It is especially relevant for classification problems [19].

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}$$

Where

- TP (True Positives) is the number of correctly predicted positive instances.

- TN (True Negatives) is the number of correctly predicted negative instances.
- FP (False Positives) is the number of actual negative instances incorrectly predicted as positive.
- FN (False Negatives) is the number of actual positive instances incorrectly predicted as negative.

Validation Accuracy (Val_ACC), often monitored during model training, is similar to accuracy but is computed on a separate validation dataset that the model has not seen during training. It provides a measure of how well the model generalizes to unseen data.

Loss represents the error or cost associated with the model's predictions. The aim during training is to minimize this loss function. Common loss functions for classification tasks include categorical cross-entropy and binary cross-entropy.

Validation Loss (Val_Loss) is the loss calculated on the validation dataset, providing insights into how well the model generalizes.

Test Accuracy (Test_ACC) measures the model's performance on a completely new and independent dataset, similar to the validation dataset.

Confusion Matrix (CM) is a crucial tool for evaluating the performance of a classification model [20]. It provides a detailed breakdown of the model's predictions and the actual classes of instances in the dataset. The matrix is especially useful in understanding the model's behaviour in terms of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

A confusion matrix is a 2×2 matrix (for binary classification) or $n \times n$ matrix (for multiclass classification), where n is the number of classes. The general structure of the confusion matrix is as follows:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Where

- **True Positives (TP):** The number of instances correctly predicted as positive.
- **True Negatives (TN)**: The number of instances correctly predicted as negative.
- **False Positives (FP)**: The number of actual negative instances incorrectly predicted as positive (Type I error).
- **False Negatives (FN)**: The number of actual positive instances incorrectly predicted as negative (Type II error).

Confusion matrices are indispensable in model evaluation and are often used in various domains, including healthcare, finance, and natural language processing, among others.
Precision (Positive Predictive Value): Precision measures the accuracy of the positive predictions made by a model. It answers the question: Of the instances the model predicted as positive, how many were actually positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Sensitivity, True Positive Rate): Recall measures the model's ability to capture all the positive instances. It answers the question: Of all the actual positive instances, how many did the model correctly predict as positive?

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score: F1 Score is the harmonic mean of precision and recall. It is a balanced measure that considers both false positives and false negatives. A high F1 score indicates good model performance.

$$\text{F1 Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

ROC-AUC Score (Receiver Operating Characteristic - Area Under the Curve): ROC-AUC measures the ability of the model to distinguish between positive and negative instances. It quantifies the area under the receiver operating characteristic (ROC) curve, which plots the true positive rate against the false positive rate across different thresholds.

5.3 Result Description

The performance analysis of these machine learning algorithms, based on metrics such as precision, recall, F1 score, and accuracy, provides valuable insights into their capabilities for classification tasks.

The Convolutional Neural Network (CNN) and Gated Recurrent Unit (GRU) showcase relatively balanced performances. CNN demonstrates consistent and even performance across the metrics, maintaining precision, recall, F1 score, and accuracy around the 70-71% range. Similarly, GRU maintains reasonable accuracy, albeit with slightly lower precision, recall, and F1 score averaging around 66-67%. In contrast, the Bidirectional LSTM (BiLSTM) presents an intriguing trade-off. It achieves a higher recall rate of around 65% but at the expense of precision, which is notably lower at approximately 58%. This indicates BiLSTM's proficiency in capturing a larger number of true positive instances while also generating more false positives compared to the other models. The standout performer among these algorithms is the LSTM model augmented with BERT. It attains an impressive precision of 81%, a recall of 76%, and an F1 score of 78%, showcasing a superior ability to accurately identify positive instances while maintaining a balanced overall performance shown in Figure 5.1.

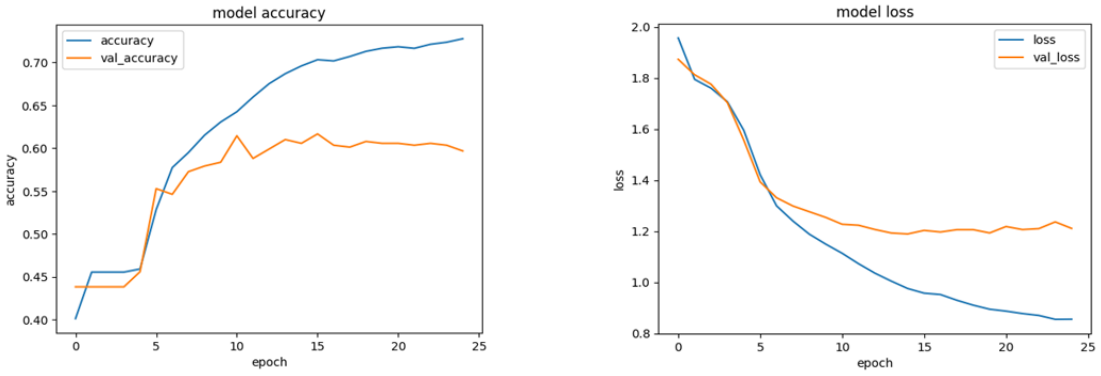

Figure 5.1: Model Loss & Accuracy.

In conclusion, while CNN and GRU demonstrate consistent and comparable performance, BiLSTM offers a trade-off with higher recall but lower precision. The LSTM model incorporating BERT emerges as the top performer, excelling in precision, recall, and F1 score. The selection of the most suitable algorithm should be driven by the specific requirements of the application, whether it prioritizes balanced performance,

higher recall, or superior precision in classification tasks.

Bi-LSTM: The Bidirectional LSTM (BiLSTM) presents a trade-off between precision and recall, with higher recall (around 65%) but notably lower precision (approximately 58%) compared to CNN and GRU. This indicates its capability to capture more positive instances but at the expense of generating more false positives shown in Figure 5.2.

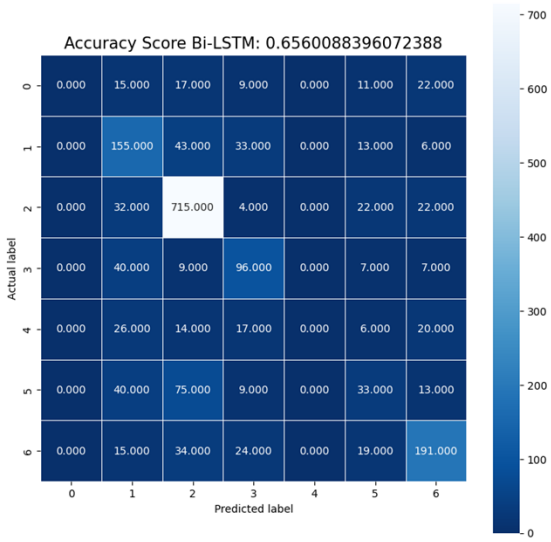

Figure 5.2: Accuracy Score For Bi-LSTM Confusion Matrix.

CNN The Convolutional Neural Network (CNN) demonstrates balanced performance across metrics, with precision, recall, F1 score, and accuracy hovering around 70-71%. This suggests its consistent ability to accurately identify both positive and negative instances shown in Figure 5.3.

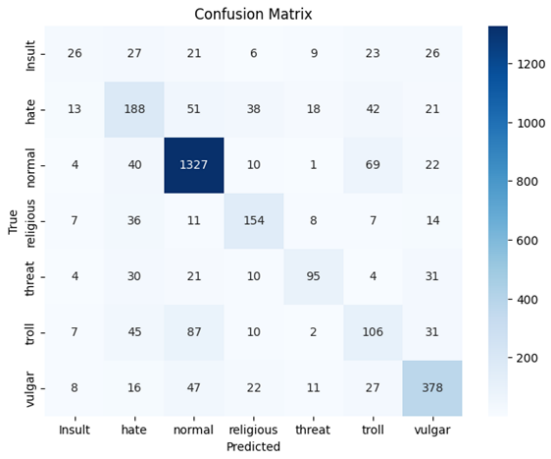

Figure 5.3: Confusion Matrix For CNN.

GRU Gated Recurrent Unit (GRU) shows slightly lower performance compared to the

CNN, with precision, recall, and F1 score averaging around 66-67%. While still maintaining reasonable accuracy, the GRU struggles a bit more in correctly identifying true positives shown in Figure 5.4.

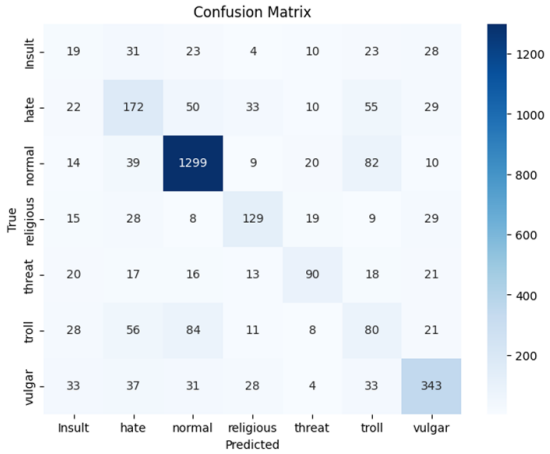

Figure 5.4: Confusion Matrix For GRU.

LSTM & BERT The LSTM model augmented with BERT demonstrates superior performance, achieving an 81% precision, 76% recall, and 78% F1 score. This model exhibits the best balance between correctly identifying positive instances and minimizing false positives among the listed algorithms. Loss and accuracy curve shown in Figure 5.5.

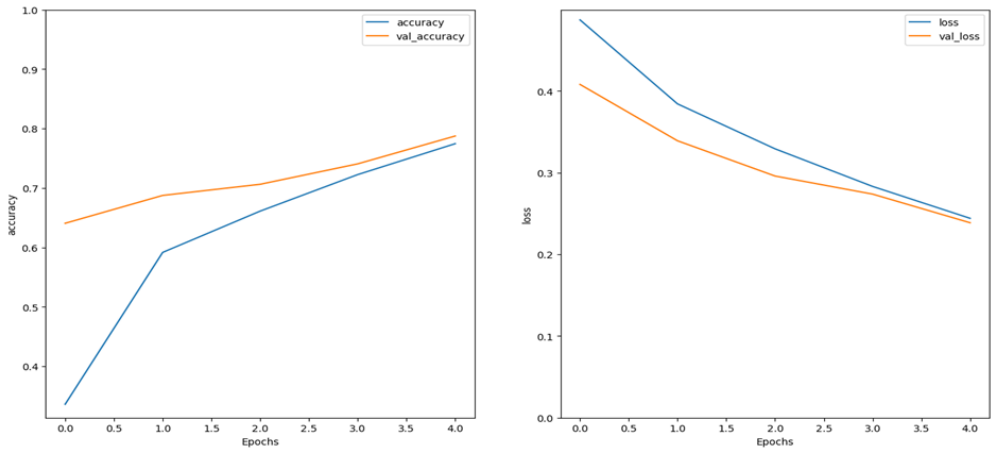

Figure 5.5: Accuracy & Loss Curve For LSTM .

While CNN and GRU exhibit balanced performance, BiLSTM offers higher recall but at the cost of precision. The LSTM with BERT stands out for achieving the best overall balance between precision and recall. The choice of algorithm should align with specific

application requirements, considering the trade-offs between precision, recall, and the need for balanced performance.

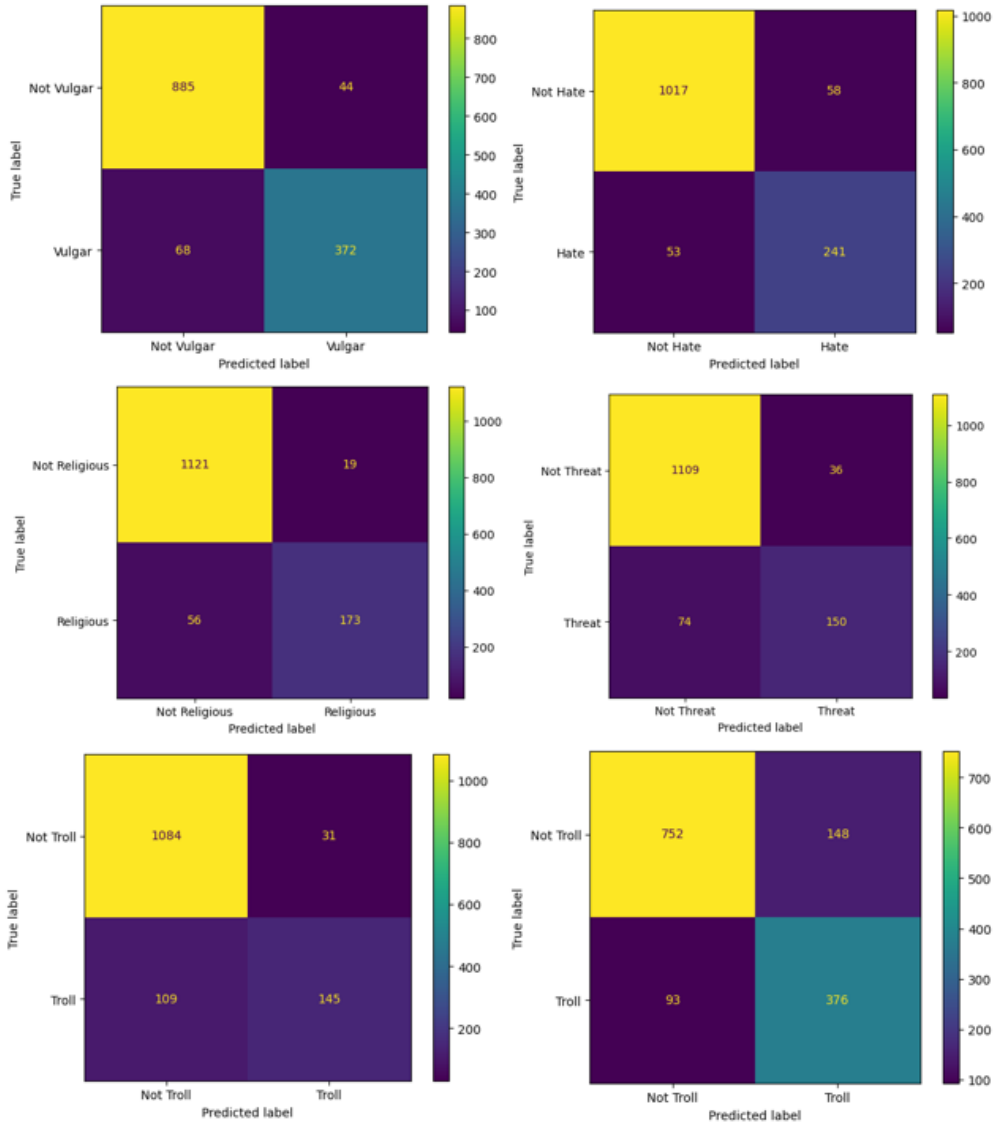

Figure 5.6: Confusion Matrix LSTM.

5.4 Experimental/Simulation Results

Firstly, the micro-average ROC curve, boasting an area of 0.94, consolidates performance by considering aggregated true positive and false positive rates across all classes. Secondly, the macro-average ROC curve, with an area of 0.93, provides a balanced evaluation across individual class ROC curves shown in Figure 5.7, offering insights into the model's overall performance across various classes. Moreover, individual ROC curves

Table 5.1: Performance Metrics of Different Models

Algorithm	Precision	Recall	F1 Score	Accuracy
LSTM+BERT	81%	76%	78%	75%
CNN	70%	71%	70%	71%
GRU	67%	66%	63%	66%
BiLSTM	58%	65%	62%	65%

for each class demonstrate discriminative ability, displaying areas ranging from 0.90 to 0.95. These specific class ROC curves indicate how well the model distinguishes each class from the rest. Higher areas under the ROC curve (AUC) generally signify better model performance, showcasing the model's strong discriminative capability across multiple classes. Overall, these metrics collectively highlight the model's proficiency in handling diverse class distinctions in the multi-class classification task. Result abalysis shown in Table 5.1 and confusion matrics show in Figure 5.6.

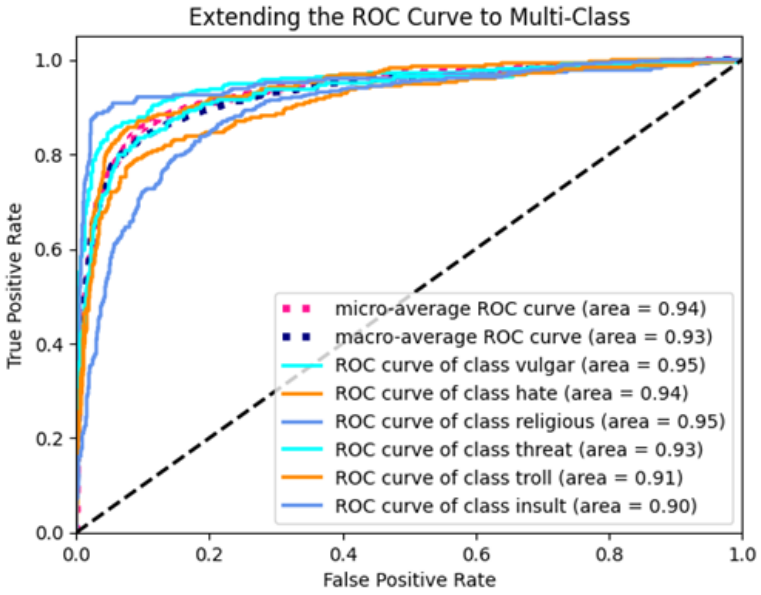

Figure 5.7: ROC Curve For Multiclass.

5.5 Performance/Comparative Analysis

Here we represent a Table 5.2 with comparing with other research works deliberately done by researchers. "Haque [24]" exhibited 58% precision, 51% recall, 48% F1 score, and 61% accuracy. "Mahimul Islam [25]" demonstrated 48% precision, 52% recall, 62% F1

Table 5.2: Comparative Result Analysis

Other Works	Precision	Recall	F1 Score	Accuracy
Haque [24]	58%	51%	48%	61%
Mahimul Islam [25]	48%	52%	62%	59%
Kamal Sarkar [26]	62%	57%	62%	61%
LSTM+BERT	81%	76%	78%	75%

score, and 59% accuracy. ”Kamal Sarkar [26]” showcased 62% precision, 57% recall, 62% F1 score, and 61% accuracy. Notably, the ”LSTM+BERT” approach achieved substantially higher metrics across the board, boasting 81% precision, 76% recall, 78% F1 score, and 75% accuracy. This comparison highlights the superior performance of the ”LSTM+BERT” model in this specific task compared to the other referenced methodologies or studies.

5.6 Summary

As for the conclusion of our grand indenture in this research work, we hereby declare that the best outcome was achieved by the LSTM&BERT. The accuracy was achieved by 75%, the f1score was 78%, recall 76% and the precision was 81%. In overall comparison the output was achieved by the conventional tuning of the transfer learning procedure as shown in the methodology part. There was also a deliberate comparison between other research works shown. But as for the accuracy achievement, we further more want to improve it by the future work adversity.

CHAPTER 6

Conclusions and Future Work

6.1 Conclusions

In the current era, there is a need for an efficient way to categorise Bangla text, and the primary objective of this research project is to create a model for doing just that. And by leveraging deep learning technologies, we were able to accomplish this with exceptional accuracy. The identical text data are divided into various categories in this dataset. Models were incorrectly classified and unable to learn successfully as a result. The suggested hybrid model outperformed other single deep neural network models in terms of performance, earning an accuracy of 75% across seven categories. The categorization of texts is a very difficult task because there aren't enough trustworthy sources or standard, balanced datasets available in the Bangla language. If there are sufficient Bangla language resources available, these models for deep learning can yield better results than this one. Future research may therefore concentrate on producing a uniform balanced Bangla dataset in addition to creating a more effective model for this NLP text categorization issue.

6.2 Future Recommendation

Supervised knowledge is utilised to achieve Bengali document categorization in this instance. Therefore, there remains more research experiments that need to be conducted without supervision. Other than the currently published works, such as cosine similarity measures, can be investigated through the use of similarity metrics. The size of the dataset should be expanded in the future to create a sophisticated and uniform output measurement system. Coherent, intricate, and larger datasets will require analysis using improved kernels that effectively perform the classification tasks.

6.3 Limitations

The main limitation we faced was the limited resources. As Bengali NLP research is an ongoing paradigm of the research contradiction, we are hoping for the better resources will soon come in order for the improvement of the technological evolution.

REFERENCES

[1] R. Nielsen, R. Fletcher, N. Newman, J. Brennen, and P. Howard, *Navigating the 'infodemic': How people in six countries access and rate news and information about coronavirus*. Reuters Institute for the Study of Journalism, 2020.

[2] "Mobs beat five dead for 'kidnapping' — thedailystar.net," https://www.thedailystar.net/frontpage/news/mobs-beat-2-dead-kidnapping-1774471.

[3] M. Z. Hossain, M. A. Rahman, M. S. Islam, and S. Kar, "Banfakenews: A dataset for detecting fake news in bangla," *arXiv preprint arXiv:2004.08789*, 2020.

[4] T. Islam, S. Latif, and N. Ahmed, "Using social networks to detect malicious bangla text content," in *2019 1st International Conference on Advances in Science, Engineering and Robotics Technology (ICASERT)*. IEEE, 2019, pp. 1–4.

[5] linguisticsurvey, https://peopleslinguisticsurvey.org/.

[6] timesofindia, https://shorturl.at/dloqS.

[7] A. Dhar, H. Mukherjee, N. S. Dash, and K. Roy, "Automatic categorization of web text documents using fuzzy inference rule," *Sādhanā*, vol. 45, pp. 1–22, 2020.

[8] A. Dhar, N. S. Dash, and K. Roy, "Classification of bangla text documents based on inverse class frequency," in *2018 3rd International Conference On Internet of Things: Smart Innovation and Usages (IoT-SIU)*. IEEE, 2018, pp. 1–6.

[9] Dhar, "Application of tf-idf feature for categorizing documents of online bangla web text corpus," in *Intelligent Engineering Informatics: Proceedings of the 6th International Conference on FICTA*. Springer, 2018, pp. 51–59.

[10] A. Dhar, N. S. Dash, and K. Roy, "Categorization of bangla web text documents based on tf-idf-icf text analysis scheme," in *Social Transformation–Digital Way: 52nd Annual Convention of the Computer Society of India, CSI 2017, Kolkata, India, January 19-21, 2018, Revised Selected Papers 52*. Springer, 2018, pp. 477–484.

[11] K. Sarkar, "Bengali text summarization by sentence extraction," *arXiv preprint arXiv:1201.2240*, 2012.

[12] P. K. Panigrahi and N. Bele, "A review of recent advances in text mining of indian languages," *International Journal of Business Information Systems*, vol. 23, no. 2, pp. 175–193, 2016.

[13] T. Thaipisutikul, S. Tuarob, S. Pongpaichet, A. Amornvatcharapong, and T. K. Shih, "Automated classification of criminal and violent activities in thailand from online news articles," in *2021 13th International Conference on Knowledge and Smart Technology (KST)*. IEEE, 2021, pp. 170–175.

[14] M. Helal and M. Mouhoub, "Topic modelling in bangla language: An lda approach to optimize topics and news classification," *Computer and Information Science*, vol. 11, no. 4, pp. 77–83, 2018.

[15] O. Barabash, O. Laptiev, O. Kovtun, O. Leshchenko, K. Dukhnovska, and A. Biehun, "The method dynavic tf-idf," *International Journal of Emerging Trends in Engineering Research*, vol. 8, no. 9, pp. 5712–5718, 2020.

[16] P. K. Panigrahi and N. B. A. review of recent advances in text mining of Indian languages." International Journal of Business Information Systems 23.2 (2016): 175-193, "The method dynavic tf-idf," *International Journal of Emerging Trends in Engineering Research*, vol. 8, no. 9, pp. 5712–5718, 2020.

[17] Q. I. Mahmud, N. I. Chowdhury, and M. Masum, "Reducing feature space and analyzing effects of using non linear kernels in svm for bangla news categorization," in *2018 International Conference on Bangla Speech and Language Processing (ICBSLP)*. IEEE, 2018, pp. 1–6.

[18] C. M. Bishop and N. M. Nasrabadi, *Pattern recognition and machine learning*, 2006, vol. 4, no. 4.

[19] A. Géron, *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*. " O'Reilly Media, Inc.", 2022.

[20] J. T. Townsend, "Theoretical analysis of an alphabetic confusion matrix," *Perception & Psychophysics*, vol. 9, pp. 40–50, 1971.

APPENDIX A

Import Necessary Library

```
pip install numpy pandas scikit-learn keras
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
import numpy as np
from tensorflow import keras
import numpy as np
from tensorflow import keras
from tensorflow.keras.layers import Dense, Activation
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.metrics import categorical_crossentropy
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import Model
from tensorflow.keras.applications import imagenet_utils
from sklearn.metrics import confusion_matrix
import itertools
import os
import shutil
import random
import matplotlib.pyplot as plt
%matplotlib inline
```

Read dataset and Define Path

```
df = pd.read_csv("/content/drive/MyDrive/Bengali word assimilation integrating deep learning & machine learning conciliar./cleaned.csv")
```

Preprocess Data

```
df['cleaned'].fillna('', inplace=True)

texts = df['cleaned'].values
labels = df['tag'].values

label_encoder = LabelEncoder()
labels = label_encoder.fit_transform(labels)
num_classes = len(np.unique(labels))

texts_train, texts_test, labels_train, labels_test = train_test_split(texts, labels, test_size=0.2, random_state=42)

texts_train
```

Build CNN Model

```
max_words = 10000
tokenizer = Tokenizer(num_words=max_words, oov_token="¡OOV¿")
tokenizer.fit_on_texts(texts_train)

sequences_train = tokenizer.texts_to_sequences(texts_train)
sequences_test = tokenizer.texts_to_sequences(texts_test)

max_len = 100
X_train = pad_sequences(sequences_train, maxlen=max_len, padding='post', truncating='post')
X_test = pad_sequences(sequences_test, maxlen=max_len, padding='post', truncating='post')

y_train = to_categorical(labels_train, num_classes=num_classes)
y_test = to_categorical(labels_test, num_classes=num_classes)

embedding_dim = 50
num_filters = 64
filter_size = 3

model = Sequential()
model.add(Embedding(input_dim=max_words, output_dim=embedding_dim, input_length=max_len))
model.add(Conv1D(num_filters, filter_size, activation='relu'))
model.add(GlobalMaxPooling1D())
model.add(Dense(num_classes, activation='softmax'))
```

Compile and Train CNN model

```
custom_optimizer = Adam(learning_rate=0.001)
model.compile(optimizer=custom_optimizer, loss='categorical_crossentropy', metrics=['accuracy'])

batch_size = 32
epochs = 10
model.fit(X_train, y_train, batch_size=batch_size, epochs=epochs, validation_split=0.2)

loss, accuracy = model.evaluate(X_test, y_test)
print(f'Test Loss: loss, Test Accuracy: accuracy')

y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

conf_mat = confusion_matrix(y_true, y_pred_classes)

class_names = label_encoder.classes_ print(classification_report(y_true, y_pred_classes, target_names=class_names))

plt.figure(figsize=(8, 6))
sns.heatmap(conf_mat, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```

GRU MODEL

```
max_words = 10000
max_len = 50
tokenizer = Tokenizer(num_words=max_words, oov_token="¡OOV¿")
tokenizer.fit_on_texts(df['cleaned'])
X = tokenizer.texts_to_sequences(df['cleaned'])
X = pad_sequences(X, maxlen=max_len)
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(df['tag'])
y_one_hot = to_categorical(y, num_classes=len(label_encoder.classes_))
X_train, X_test, y_train, y_test = train_test_split(X, y_one_hot, test_size=0.2, random_state=42)
def lr_scheduler(epoch, lr):
if epoch % 3 == 0 and epoch= 0:
return lr * 0.5
return lr
model = Sequential()
model.add(Embedding(input_dim=max_words, output_dim=64, input_length=max_len))
model.add(GRU(64, dropout=0.2, recurrent_dropout=0.2, return_sequences=True))
model.add(GRU(32, dropout=0.2, recurrent_dropout=0.2))
model.add(Dense(32, activation='relu'))
model.add(Dense(len(label_encoder.classes_), activation='softmax'))
model.compile(optimizer=Adam(learning_rate=0.001),
loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2, callbacks=[LearningRateScheduler(lr_scheduler)])
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Test Loss: loss:.4f, Test Accuracy: accuracy:.4f')
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)
conf_mat = confusion_matrix(y_true, y_pred_classes)
class_names = label_encoder.classes_
print(classification_report(y_true, y_pred_classes, target_names=class_names))
plt.figure(figsize=(8, 6))
sns.heatmap(conf_mat, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```

Preprocessing Bi-LSTM MODEL

```
def stopwordRemoval(text):
x = str(text)
l = x.split()
stm = [elem for elem in l if elem not in stop]
out = ' '.join(stm)
return str(out)
data1 = pd.read_excel('/content/drive/MyDrive/Code/document/stopwords_bangla.xlsx')
display(data1)
stop = data1['words'].tolist()
!pip install bangla-stemmer
from bangla_stemmer.stemmer import stemmer
def stem_text (x):
stmr = stemmer.BanglaStemmer()
words=x.split(' ')
stm = stmr.stem(words)
words=(' ').join(stm)
return words
df_train["tag"].replace("Insult": "0", "hate": "1", "normal": "2", "religious": "3",
"threat": "4", "troll": "5", "vulgar": "6", inplace=True)
df_test["tag"].replace("Insult": "0", "hate": "1", "normal": "2", "religious": "3",
"threat": "4", "troll": "5", "vulgar": "6", inplace=True)
display(df_train)
display(df_test)
df_train = df_train.dropna()
df_test = df_test.dropna()
df_train['count'] = df_train['cleaned'].str.split().str.len()
df_test['count'] = df_test['cleaned'].str.split().str.len()
df_train= df_train.loc[df_train['count']¿5]
df_test= df_test.loc[df_test['count']¿5]
display(df_train)
display(df_test)
df_train = df_train.sample(frac=1).reset_index(drop=True)
display(df_train)
display(df_test)
train_sentences=df_train['cleaned'].values
train_labels=df_train['tag'].values
test_sentences=df_test['cleaned'].values
test_labels=df_test['tag'].values
print("Training Set Length: "+str(len(df_train)))
print("Testing Set Length: "+str(len(df_test)))
print("train_labels shape: "+str(train_labels.shape)))
print("test_labels shape: "+str(test_labels.shape)))
```

Build model

```
test_sentences, validation_sentences,test_labels, validation_labels =
train_test_split(test_sentences, test_labels,stratify=test_labels,test_size=0.2)
train_labels = keras.utils.to_categorical(train_labels)
test_labels = keras.utils.to_categorical(test_labels)
validation_labels = keras.utils.to_categorical(validation_labels)
print(”Training Set Length: ”+str(len(df_train)))
print(”Testing Set Length: ”+str(len(df_test)))
print(”training_sentences shape: ”+str(train_sentences.shape)))
print(”testing_sentences shape: ”+str(test_sentences.shape)))
print(”validation_sentences shape: ”+str(validation_sentences.shape)))
print(”train_labels shape: ”+str(train_labels.shape)))
print(”test_labels shape: ”+str(test_labels.shape)))
print(”validation_labels shape: ”+str(validation_labels.shape)))
print(train_sentences[1]))
print(train_labels[0]))
vocab_size = 1000
embedding_dim = 64
max_length = 200
trunc_type=’post’
oov_tok = ”¡OOV¿”
print(train_sentences.shape))
print(train_labels.shape))
tokenizer = Tokenizer(num_words = vocab_size, oov_token=oov_tok)
tokenizer.fit_on_texts(train_sentences)
word_index = tokenizer.word_index
print(len(word_index))
print(”Word index length:”+str(len(tokenizer.word_index)))
sequences = tokenizer.texts_to_sequences(train_sentences)
padded = pad_sequences(sequences,maxlen=max_length, truncating=trunc_type))
test_sequences = tokenizer.texts_to_sequences(test_sentences)
testing_padded = pad_sequences(test_sequences,maxlen=max_length))
validation_sequences = tokenizer.texts_to_sequences(validation_sentences)
validation_padded = pad_sequences(validation_sequences,maxlen=max_length))
print(”Sentence :–¿
n”))
print(train_sentences[2]+””))
print(”Sentence Tokenized and Converted into Sequence :–¿
n”))
print(str(sequences[2])+””))
```

Train Model

```
print(”After Padding the Sequence with padding length 100 :–¿
n”))
print(padded[2]))
print(”Padded shape(training): ”+str(padded.shape))
print(”Padded shape(testing): ”+str(testing_padded.shape))
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense, Flatten
from tensorflow.keras import regularizers
from tensorflow.keras.optimizers import Adam
model = Sequential()
model.add(Embedding(vocab_size, embedding_dim, input_length=max_length))
model.add(Bidirectional(LSTM(16, return_sequences=True)))
model.add(Dense(16, kernel_regularizer=regularizers.l2(0.01), activation=”relu”))
model.add(Flatten())
model.add(Dense(7, activation=’softmax’))
adam = Adam(learning_rate=0.0005, beta_1=0.9, beta_2=0.999, epsilon=1e-07, ams-
grad=False))
model.summary()
model.compile(loss=’categorical_crossentropy’, optimizer=adam, met-
rics=[’accuracy’]))
history = model.fit(
padded,
train_labels,
epochs= 25,
batch_size=512,
validation_data=(validation_padded, validation_labels),
use_multiprocessing=True,
workers=8))
print(history.history.keys())
loss = history.history[’loss’]
val_loss = history.history[’val_loss’]
```

LSTM+BERT

```
from keras import initializers as initializers, regularizers, constraints
REG_PARAM = 1e-13
def bert_model():
bert_encoder = TFBertModel.from_pretrained("sagorsarker/bangla-bert-base")
input_word_ids = tf.keras.Input(shape=(max_length,), dtype=tf.int32,
name="input_ids")
attn_masks = tf.keras.Input(shape=(max_length,), dtype=tf.int32,
name="attention_mask")
encoder_embedding_layer = bert_encoder(input_word_id.
attention_mask=attn_masks)[0]
conv1D_1 = tf.keras.layers.Conv1D(512, 4,
activation='relu',name='con1')(encoder_embedding_layer)
maxPool1D_1 =
tf.keras.layers.MaxPooling1D(pool_size=2,name='maxpool1')(conv1D_1)
conv1D_2 = tf.keras.layers.Conv1D(256, 3,
activation='relu',name='con2')(maxPool1D_1)
maxPool1D_2 =
tf.keras.layers.MaxPooling1D(pool_size=2,name='maxpool2')(conv1D_2)
conv1D_3 = tf.keras.layers.Conv1D(128, 2,
activation='relu',name='con3')(maxPool1D_2)
maxPool1D_3 =
tf.keras.layers.MaxPooling1D(pool_size=2,name='maxpool3')(conv1D_3)

lstm_outs = keras.layers.LSTM(128, return_sequences=True, ker-
nel_regularizer=regularizers.l2(REG_PARAM))(maxPool1D_3)

input_dim = int(lstm_outs.shape[2])
permuted_inputs = keras.layers.Permute((2, 1))(lstm_outs)
attention_vector = keras.layers.TimeDistributed(keras.layers.Dense(1))(lstm_outs)

attention_vector = keras.layers.Reshape((attention_vector.shape[1],))(attention_vector)
attention_vector = keras.layers.Activation('softmax',
name='attention_vec')(attention_vector)

attention_output = keras.layers.Dot(axes=1)([lstm_outs, attention_vector])
fc = keras.layers.Dense(64, activation='relu')(attention_output)
output = keras.layers.Dense(6, activation='sigmoid')(fc)
model = keras.Model(inputs=[input_word_ids, attn_masks], outputs=output)
return model
model = bert_model()
model.compile(loss=tf.keras.losses.BinaryCrossentropy(from_logits=False),
optimizer=tf.keras.optimizers.legacy.Adam(1e-5),
metrics=['accuracy', fmeasure, precision, recall])
model.summary()
```

Train model

```
from tensorflow.keras.utils import plot_model
from IPython.display import Image
plot_model(model, to_file='toxic_detection_model_plot.png', show_shapes=True, show_layer_names=True)
Image(retina=True, filename='toxic_detection_model_plot.png')
import logging
logging.getLogger('tensorflow').setLevel(logging.ERROR)
history = model.fit( train_dataset, batch_size=16, epochs=5, validation_data=dev_dataset, validation_steps=20, verbose=1)

test_score = model.evaluate(test_dataset)

print('Test Loss:', test_score[0])
print('Test Accuracy:', test_score[1])
```

APPENDIX B

Complex Engineering Problems (CP) and Complex Engineering Activities (CA) Analysis

Title: Bengali Word Assimilation Integrating Using Deep Learning Methods.

Attainment of Complex Engineering Problem (CP)

S.L.	CP No.	Attainment	Remarks
1.	P1: Depth of Knowledge Required	Yes	K3 (Engineering Fundamentals): python describe in Chapter 4. Art. 4.2.
			K4 (Engineering Specialization): knowledge about Deep learning describe in Chapter 3. Art. 3.4
			K5 (Design): Methodology flow chart describe in Chapter 3 Art. 3.2
			K6 (Technology): Tensorflow, Numpy, Pandas, etc. Described on Chapter 4 Art. 4.2
			K8 (Research): Related work describe in Chapter 2 Art. 2.1
2.	P2: Range of Conflicting Requirements	Yes	Learn about Bengali Word Assimilation Integrating describe in Chapter 3 Art. 3.3 and 3.4
3.	P3: Depth of Analysis Required	Yes	Dataset describe in 3 Art. 3.2

4.	P4: Familiarity of Issues	Yes	Study about Bengali Word Assimilation as a CSE students describe in Chapter 1 and Chapter 3
5.	P5: Extent of Applicable Codes	Yes	Follow standards methodology describe on Chapter 3
6.	P6: Extent of Stakeholder Involvement and Conflicting Requirements	Yes	Technology Developer and Social Media User Chapter 1 Art. 1.6
7.	P7: Interdependence	Yes	Collecting Dataset describe in Chapter 3.

Mapping of Complex Engineering Activities (CA)

S.L.	**CA No.**	**Attainment**	**Remarks**
1.	A1: Range of resources	Yes	Technology Developer and Social Media User describe Chapter 1 Art. 1.6
2.	A2: Level of interaction	Yes	There are several issues come when we work this Thesis describe in Chapter 1 Art. 1.5
3.	A3: Innovation	Yes	Describe dataset in Chapter 3 and chapter 4.
4.	A4: Consequences for Society and the Environment	Yes	Involves Technology Developer and Social Media User describe in Chapter 1 Art. 1.6
5.	A5: Familiarity	Yes	Build a Dataset and recognition model describe in Chapter 3 and chapter 4.

BIOGRAPHY

of

Anindya Halder

Anindya Halder, a resident of Jhalokathi and born in 2000.He is the son of Biplab Halder and Kabita Mridha. Anindya recently achieved his Bachelor of Science in Computer Science and Engineering (CSE) from Bangladesh Army University of Science and Technology (BAUST), Saidpur. His academic journey includes obtaining his SSC in 2017 from Aourabunia Model High School, Kathalia, and his HSC in 2019 from Betagi Govt College, Barguna.

His proficiency extends far beyond academics. Anindya has cultivated expertise in Java and Python programming languages, demonstrating a strong command over both. His skills also delve into the realm of data science, where he exhibits commendable proficiency. Notably, he possesses a solid understanding of HTML and CSS, showcasing a versatile skill set spanning software development, data analysis, and web technologies.

Anindya Halder

+8801714444280

anindyahalder2000@gmail.com

Jhalokathi, Barishal, Bangladesh

BIOGRAPHY

of

Mst. Faujia Akter

Mst. Faujia Akter who now resides in Bogura, was born in 2000. Md. Abu Bakar Siddique is her father and Mst. Parul Siddique is her mother. Recently she passed her Bachelor of Science in Computer Science and Engineering (CSE) from Bangladesh Army University of Science and Technology (BAUST), Saidpur. She earned her SSC in 2017 from Bogura Cantonment Board High School, Bogura and her HSC in 2019 from Bogura Cantonment Public School and College, Bogura

She earned a certificate for passing the exam CERTIFIED APPSEC PRACTITIONER (CPA) with merit, date 04-02-2023, certificate id 7028636.

She earned a certificate for successfully completing the STEM Empowerment Boot-camp. Here she earned knowledge about Emotional Intelligence, Web Fundamental: HTML, CSS and JavaScript, Data analysis and visualization, Al mastery box for work efficiency. Could Computing and Cyber security fundamental

---

Mst. Faujia Akter

+88 01772061760

Faujia.baust@gmail.com

Sonatola,Bogura,Rajshahi,Bangladesh

BIOGRAPHY

of

Most. Joiariya Zannath

Most. Joiariya Zannath who now resides in Gaibandha, was born in 2001. Md. Jafirul Islam is her father and Mst. Fatema Kaniz is her mother. She is about to be graduated from Bangladesh Army University of Science and Technology (BAUST), Saidpur Nilphamari. Recently she pushed her Bachelor of Science in Computer Science and Engineering (CSE). She earned her SSC in 2017 and her HSC in 2019.She has completed the Computer Security, Digital Image Processing, Data Warehouse and Data Mining, Artificial Nural Networks and Fuzzy Logic system and Machine Learning course. She has a Passion for Machine Learning and Deep Learning.

Most. Joiariya Zannath

+8801770369760

Joiariyazannath22@gmail.com

Tulshighat, Gaibandha, Rangpur, Bangladesh